

High-Level Design

Learning Analysis Tool — analyzing what was taught vs. what was understood

1. Problem

Teachers deliver a session; students absorb only part of it. There is no fast, objective way to see the gap between what was **taught** and what each student **understood**. This tool closes that gap by comparing the session's concepts against the evidence students leave in the class chat.

2. Solution overview

The tool ingests two inputs — the lesson content and the Google Meet chat — and produces a per-student understanding report plus a class-wide view of weak spots. An LLM performs the semantic work (concept extraction and understanding judgement); deterministic Java code handles parsing, scoring, and presentation.

3. Architecture

A layered Spring Boot application. Each stage has one responsibility, so components are independently testable and swappable.

Layer	Component	Responsibility
Input	WordExtractor (POI)	Read teacher .docx into text
Input	ChatParser (regex)	Group Meet chat by student
Intelligence	LlmClient + Prompts	Concept extraction & understanding judgement
Analysis	Summarizer	Coverage %, gaps, strengths, class weak spots
Presentation	Controller + Thymeleaf	Upload UI, dashboard, Excel export

4. Data flow

1. Lesson file → WordExtractor / transcript → raw text.
2. Text → LLM → structured concept list (the ground truth).
3. Meet chat → ChatParser → per-student message bundles.
4. Concepts + each bundle → LLM → per-concept understanding level with evidence.
5. Assessments → Summarizer → coverage %, gaps, strengths, class weak spots.
6. Results → dashboard + Excel export.

Concept data model

Each concept carries an id, name, type (definition / process / application), atomic key points (the checkable facts), and importance (core / supplementary). Student assessments reference concepts by id, keeping comparison clean.

5. Understanding levels

Understanding is graded on a four-point scale rather than a binary mention check — this is what makes the analysis meaningful rather than keyword matching.

Level	Score	Meaning
absent	0	No chat evidence of the concept
partial	1	Mentioned but shallow or confused
understood	2	Explained correctly
mastery	3	Applied or questioned deeply

Coverage % = $\text{sum of levels} \div (\text{concepts} \times 3)$. A concept at absent or partial is a **gap**; understood or mastery is a **strength**.

6. Technology stack

- **Java 21 + Spring Boot 3** — application framework, REST + server-rendered UI
- **Apache POI** — Word (.docx) reading and Excel (.xlsx) export
- **java.net.http.HttpClient** — direct LLM API calls, no SDK dependency
- **Jackson** — JSON parsing of LLM output into typed records
- **Thymeleaf** — dashboard rendering
- **Whisper** (external) — Meet audio → transcript when no Word file exists
- **JUnit 5** — test suite

7. Meet integration

When a Google Meet is recorded on a Workspace plan, Google saves both the recording and the chat log to Drive. The chat file (timestamp, name, message per line) is the student-evidence input. For lesson content, either a teacher's Word doc is used directly, or the recording's audio is transcribed with Whisper into the same text input. The tool runs post-session, which keeps it simple and reliable.

8. Design decisions & trade-offs

- **LLM for semantics, code for scoring** — keeps results explainable and testable.
- **Four-point understanding scale** — distinguishes 'mentioned' from 'understood'.
- **Chat as evidence** — zero extra student effort, but a partial signal (see limits).
- **Post-session, not real-time** — simpler, sufficient, avoids streaming complexity.
- **Human-confirmable concepts** — extraction can be reviewed before comparison.

9. Known limitations

Chat is a partial signal: a quiet student is not necessarily a lost one, so *absent* means 'no chat evidence', not 'no understanding'. Transcription can mishear technical terms. Both are documented and can be mitigated by pairing chat with a short post-session quiz and a domain glossary for the transcriber.